

Describing Code-Generated Visuals for Computer Science (CS) teaching materials

1. Introduction

1.1 Purpose of the Guideline

Computer Science education increasingly relies on visual materials such as graphs, diagrams, plots, and code-generated images. These materials are often important for understanding computational structures, algorithms, mathematical relationships, and software systems. However, many of these visuals are inaccessible or only partially accessible to blind and visually impaired students.

This guideline provides a framework for creating accessible visual materials used in Computer Science education. The objective is not only to describe images, but also to help students understand the ideas, relationships, and concepts represented in the visualizations. Accessible materials should support independent learning, reasoning, and participation in technical subjects.

This guideline is intended for university teachers, teaching assistants, researchers, instructional designers, and students who prepare teaching materials in Computer Science and related fields.

The document focuses mainly on visuals that are created using programming code. These include graphs, plots, computational diagrams, algorithm visualizations, and other technical figures commonly used in Computer Science courses.

1.2 Scope

The guideline covers accessibility approaches for visual materials used across many Computer Science subjects, including mathematics, data science, machine learning, programming, software engineering, databases, cybersecurity, networks, web technologies, human-computer interaction, and computer architecture.

The document covers:

- alternative text,
- semantic descriptions,
- structured textual representations,
- and accessibility-aware generation of computational graphics.

The main emphasis is on helping educators create materials that communicate meaning clearly to both sighted and non-visual learners.

2. General Framework for Describing Code-Generated Visuals

2.1 What are Code-Generated Visuals?

Code-generated visuals are graphs, diagrams, images, or plots that are created fully or partially using programming code instead of manual drawing tools. In Computer Science education, these visuals are commonly produced using plotting libraries, graph visualization frameworks, simulation software, machine learning tools, and web technologies.

Examples include:

- graphs generated with Matplotlib or ggplot2,
- network visualizations created with D3.js or NetworkX,
- UML diagrams generated from source code,
- plots rendered from numerical simulations,
- computational graphs from machine learning frameworks,
- and browser-rendered SVG or Canvas graphics.

Unlike decorative images, code-generated visuals usually represent structured information. They often show relationships between variables, computational processes, algorithm behavior, or mathematical patterns. Because of this, accessibility should focus not only on what the visual looks like, but also on what it means and what students are expected to learn from it.

2.2 Objectives of Accessible Code-Generated Visuals

Accessible code-generated visuals should support three primary goals:

- **Orientation:** Helping students understand what type of visual is presented and how it is organized. This includes understanding what variables are shown, what the axes represent, and what the overall purpose of the visualization is.
- **Interpretation:** Helping students understand relationships, patterns, behaviors, and computational meaning. Students should be able to interpret the information rather than simply hear a description of shapes or colors.
- **Independent Analysis:** Accessible visuals should support independent analysis. Students should be able to reason about the data, compare results, identify patterns, and solve problems without depending entirely on visual interpretation.

These objectives should be followed regardless of programming language or library used for creating visuals.

3. Programming Languages and Code-Based Graphics

Modern Computer Science teaching materials frequently rely on programming languages and software libraries to create visualizations dynamically. These visuals may represent datasets, algorithms, simulations, software systems, or mathematical behavior. Many are interactive, automatically generated, or connected directly to computational workflows.

Because these visuals are generated from code, accessibility should not focus only on the final image. It is also important to consider how the visualization is constructed, what data it represents, and how students can access the underlying information.

This document provides guidance for generating visuals using common programming languages and graphics frameworks used in Computer Science education. Regardless of programming language or framework, accessible code-generated visuals should:

1. Prioritize Semantic Meaning

Accessible visualizations should communicate meaning clearly, including relationships between variables and their exhibited behaviors.

2. Preserve Computational Context

Students should understand how the graph relates to the code, what variables are represented, and what process generated the visual output.

3. Support Reproducibility

In many situations, providing access to the source code itself can improve accessibility because it allows students to inspect the logic and structure behind the visualization.

Where relevant, descriptions should reference:

- input data,
- algorithms,
- parameters,
- and transformations.

4. Support Non-Visual Exploration

Where possible, educational materials should also include structured data tables, textual summaries, keyboard-accessible interactions, and downloadable datasets. Blind students often benefit from direct access to the underlying data instead of relying entirely on visual interpretation.

Accessibility should therefore be treated as part of the visualization design process rather than something added afterward.

4. Accessibility-Aware Plot Design

Accessible plots should be designed with accessibility in mind from the beginning. Instead of creating a graph first and adding accessibility later, the plotting code itself should support readability, interpretation, and non-visual access.

Accessibility-aware plotting improves readability, interpretability, screen-reader compatibility, keyboard navigation, and multimodal access. The following recommendations apply across plotting libraries and programming languages.

4.1 Axis Labels

One of the most important practices is using clear and meaningful axis labels. Axes should always include descriptive labels, measurement units where appropriate, and meaningful variable names.

Labels such as “x” or “value” provide little information to students using screen readers and should be avoided. Labels should explain what is being measured and should include units where appropriate. For example, labels such as “Training Epoch,” “Execution Time (ms),” or “Study Time (hours)” communicate much more useful information.

Example:

```
plt.xlabel("Training Epoch")
plt.ylabel("Validation Accuracy (%)")
```

Axis labels are particularly important because screen readers and accessibility tools frequently extract them automatically.

4.2 Titles

Plot titles should also explain the purpose of the graph. A good title helps students understand immediately what relationship or process is being shown.

Generic titles such as “Graph 1” and “Results” should be avoided. Instead, the title should describe the content of the visualized graph: e.g., “Validation Accuracy During Neural Network Training”, “Quadratic Growth of Algorithm Runtime”.

4.3 Font Size

Readable font sizes are important for low-vision users and projected lecture materials. Small text can become difficult to read when students use screen magnification tools or attend lectures in large classrooms. Increasing font size improves readability significantly.

Recommended minimum font sizes are 12-14 pt for embedded figures, and 16+ pt for presentations.

Example:

```
plt.xlabel("Epoch", fontsize=14)
plt.ylabel("Loss", fontsize=14)
plt.title("Training Loss Curve", fontsize=16)
```

4.4 Accessible Color Palettes

Color choices also strongly affect accessibility. Color should never be the only method used to communicate information because some students may have color vision deficiencies or may access the plot non-visually. High-contrast color palettes should be preferred, and different line styles or marker shapes should be used together with color to distinguish data series.

Plots should:

- remain interpretable in grayscale,
- avoid problematic color combinations,
- and support color vision deficiencies.

Things to avoid include red-green combinations, low-contrast colors, and purely decorative gradients. Instead, it is recommended to use high-contrast palettes, colorblind-friendly palettes, and redundant encodings.

Useful color palettes include:

- `tab10`
- `viridis`
- `cividis`

Example:

```
plt.plot(x, y, color="black", linestyle="--", marker="o")
```

Combining solid and dashed lines, circular and square markers, and direct labels can make plots much easier to interpret.

4.5 Grid Lines

Grid lines may also help students estimate values and follow trends more easily, although excessive grid density can make figures cluttered and difficult to read.

Example:

```
plt.grid(True)
```

4.6 Redundant Visual Encodings

Important distinctions should not rely solely on color. Instead, it is recommended to use combinations of line styles, marker shapes, labels, textures, and annotations.

Example:

```
plt.plot(x1, y1, linestyle="-", marker="o", label="Training")
plt.plot(x2, y2, linestyle="--", marker="s", label="Validation")
```

4.7 Overcrowding Avoidance

Another important principle is avoiding overcrowded plots. Figures containing too many categories, overlapping lines, or dense legends may become difficult to interpret visually and challenging for automated accessibility tools. In many situations, it is better to split a complex visualization into several simpler figures.

It is recommended to reduce clutter by:

- limiting unnecessary categories,

- splitting complex figures,
- simplifying legends,
- and highlighting important information.

4.8 Structured Data Access

Whenever possible, students should also be given access to the underlying data in structured form. Providing CSV files, accessible tables, or textual summaries allows blind students to explore the information directly and independently.

5. Graphs and Plot Types

This section provides practical guidance for creating accessible code-generated visualizations commonly used in Computer Science education. The examples use Python and Matplotlib for consistency, but the same principles apply to other plotting libraries and programming languages.

The goal of this section is not only to explain how graphs should be described, but also how they can be generated in a more accessible way directly through code. Accessibility-aware plotting improves readability, supports screen-reader tools, and helps students understand the meaning of the visualization more easily.

Each subsection includes:

- a simple plotting example,
- an improved accessibility-oriented version,
- an explanation of accessibility improvements,
- and an example interpretation of the graph from a semantic perspective.

The emphasis throughout the section is on helping students understand relationships, trends, structures, and computational meaning rather than only the visual appearance of the graph.

5.1 Line Graphs (Trends Over Continuous Data)

Line graphs can be used to represent change over a continuous variable (often time or input size), functional relationships, or trends and growth patterns.

Example:

```
import matplotlib.pyplot as plt

epochs = range(1, 11)
training_loss = [0.95, 0.82, 0.71, 0.60, 0.52, 0.45, 0.40, 0.37, 0.35, 0.34]

plt.plot(epochs, training_loss)
plt.show()
```

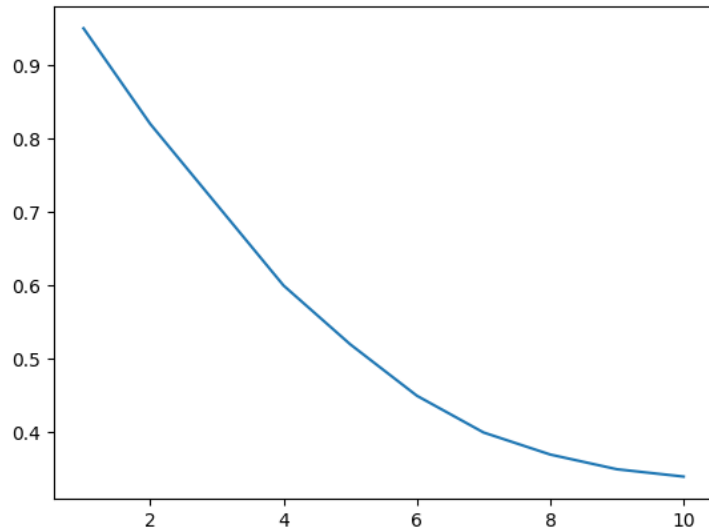


Figure 1. Inaccessible line graph showing neural-network training loss across ten epochs.

Problems identified:

- no axis labels,
- no title,
- no grid,
- low interpretability,
- inaccessible default styling,
- no semantic context.

Accessibility-Aware Version:

```
import matplotlib.pyplot as plt

epochs = range(1, 11)
training_loss = [0.95, 0.82, 0.71, 0.60, 0.52, 0.45, 0.40, 0.37, 0.35,
0.34]

plt.figure(figsize=(8, 5))

plt.plot(
    epochs,
    training_loss,
    color="black",
    linestyle="-",
    marker="o",
    linewidth=2,
    label="Training Loss"
)

plt.xlabel("Training Epoch", fontsize=14)
```

```
plt.ylabel("Loss Value", fontsize=14)
plt.title("Neural Network Training Loss Across Epochs", fontsize=16)

plt.grid(True)
plt.legend(fontsize=12)

plt.tight_layout()
plt.show()
```

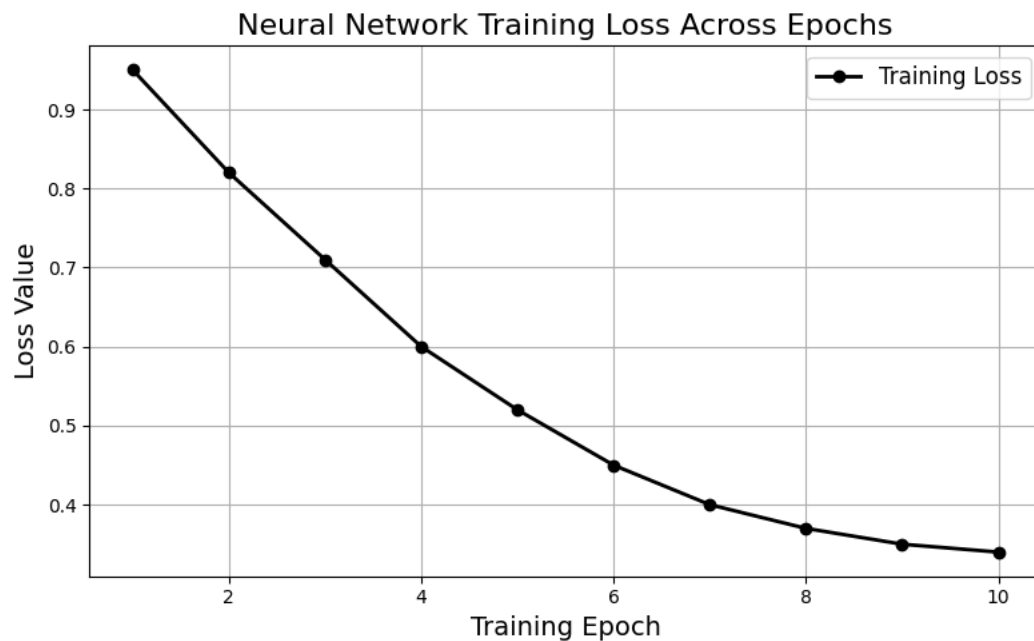


Figure 2. Accessibility-aware line graph showing neural-network training loss across ten epochs.

The improved version:

1. uses meaningful variable names,
2. includes descriptive axis labels,
3. adds a descriptive title,
4. increases font size,
5. includes grid lines,
6. uses high-contrast styling,
7. combines markers and line styles,
8. and improves readability for magnification users.

Example semantic interpretation: The line graph shows how neural network training loss changes over ten epochs. The horizontal axis represents training epochs, while the vertical axis represents loss values. The plotted line decreases rapidly during the early epochs and gradually stabilizes, indicating that the model is learning efficiently and approaching convergence.

5.2 Scatterplots (Relationships Between Variables)

Scatterplots are used to represent relationships between two variables, correlation, or clustering patterns.

Basic Example:

```
import matplotlib.pyplot as plt

x = [1, 2, 3, 4, 5]
y = [50, 60, 72, 85, 95]

plt.scatter(x, y)
plt.show()
```

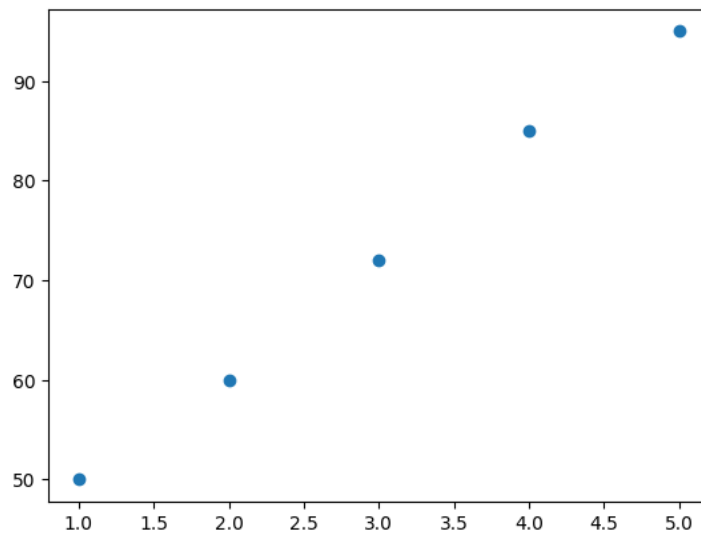


Figure 3. Basic scatterplot of five paired numerical observations without semantic labels.

Problems:

- unclear variable meaning,
- no labels,
- inaccessible default colors,
- no contextual explanation,
- no support for interpretation.

Accessibility-Aware Version:

```
import matplotlib.pyplot as plt

study_hours = [1, 2, 3, 4, 5]
exam_scores = [50, 60, 72, 85, 95]

plt.figure(figsize=(8, 5))

plt.scatter(
```

```

    study_hours,
    exam_scores,
    color="black",
    marker="o",
    s=80,
    label="Student Performance"
)

plt.xlabel("Study Time (Hours)", fontsize=14)
plt.ylabel("Exam Score (%)", fontsize=14)
plt.title("Relationship Between Study Time and Exam Performance",
          fontsize=16)

plt.grid(True)
plt.legend(fontsize=12)

plt.tight_layout()
plt.show()

```

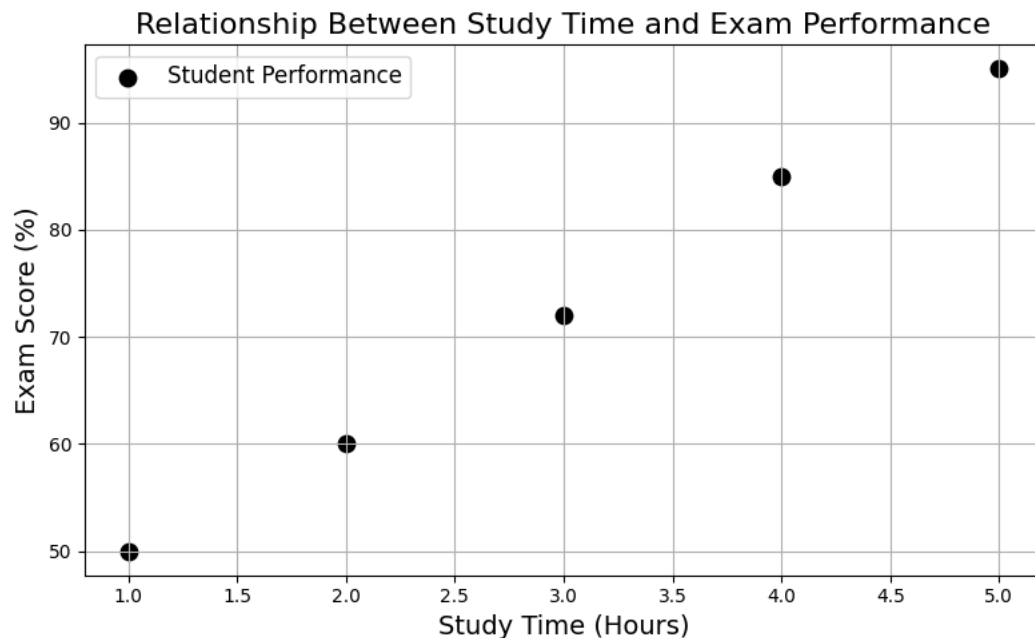


Figure 4. Accessibility-aware scatterplot relating study time to exam performance.

The improved version:

- clarifies what each axis represents,
- uses meaningful educational context,
- increases marker size,
- improves readability,
- and supports semantic interpretation.

Example semantic interpretation: Each point on the scatterplot represents one student. The horizontal axis shows study time in hours, while the vertical axis shows exam scores. The points form a strong upward trend, indicating a positive correlation between study time and exam performance. No significant outliers are present.

5.3 Histograms (Distributions)

Histograms can be used to represent frequency distributions, data spread, and skewness of a measured variable.

Basic Example:

```
import matplotlib.pyplot as plt

data = [55, 60, 62, 65, 68, 70, 72, 75, 78, 80]

plt.hist(data)
plt.show()
```

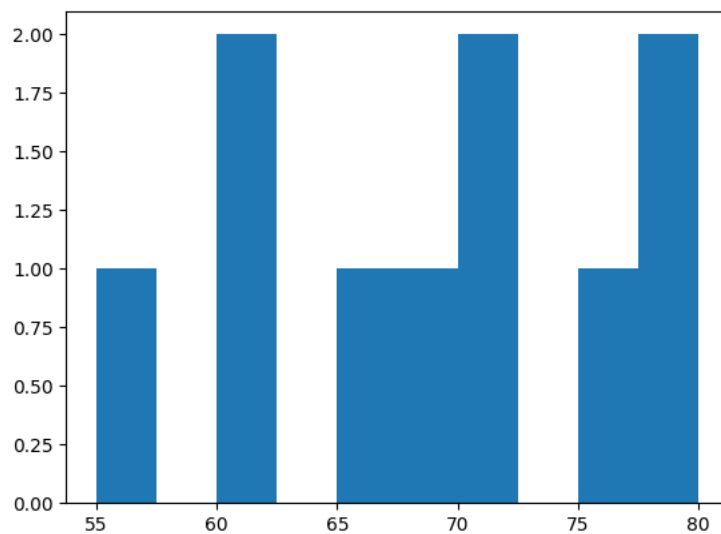


Figure 5. Basic histogram of ten exam-score values using default plotting settings.

Problems with the basic version:

- no explanation of the represented data,
- default binning may be unclear,
- missing labels and title,
- no semantic context.

Accessibility-Aware Version:

```
import matplotlib.pyplot as plt

exam_scores = [55, 60, 62, 65, 68, 70, 72, 75, 78, 80]
```

```
plt.figure(figsize=(8, 5))

plt.hist(
    exam_scores,
    bins=5,
    color="lightgray",
    edgecolor="black"
)

plt.xlabel("Exam Score (%)", fontsize=14)
plt.ylabel("Number of Students", fontsize=14)
plt.title("Distribution of Exam Scores", fontsize=16)

plt.grid(True)
plt.tight_layout()
plt.show()
```

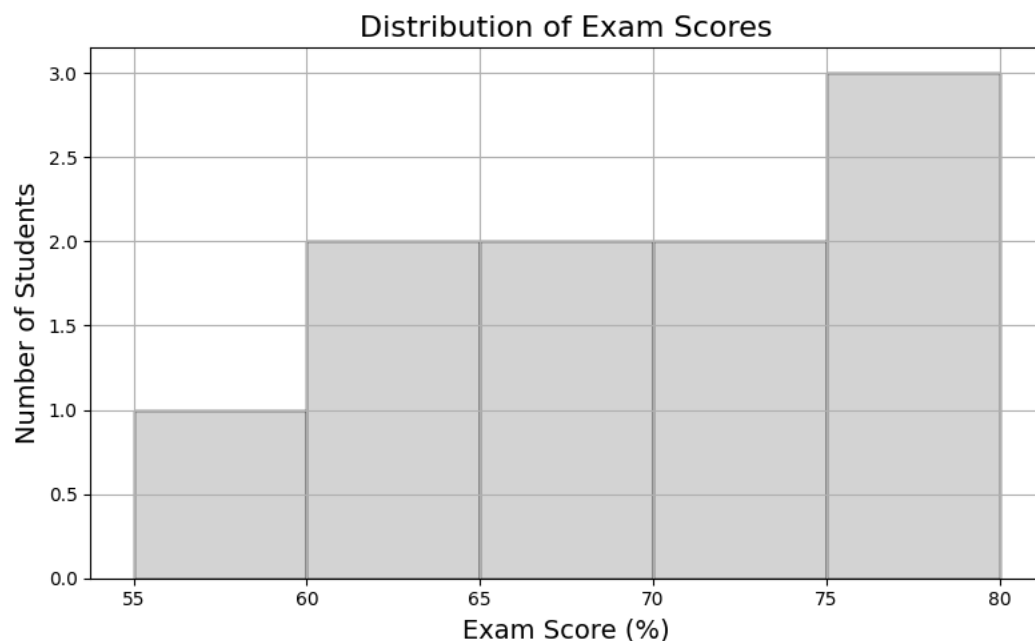


Figure 6. Accessibility-aware histogram showing the distribution of exam scores.

The improved version:

- explains the represented variable,
- adds high-contrast bin borders,
- improves readability,
- and clarifies distribution meaning.

Example semantic interpretation: The histogram represents the distribution of exam scores among students. Most scores cluster between 65 and 75 percent, indicating that the majority of

students achieved mid-to-high performance. The distribution is relatively balanced with no strong skew toward extremely low or high scores.

5.4 Box Plots (Statistical Summary)

Box plots encode the median, quartiles, variability, and outliers in the data.

Basic Example:

```
import matplotlib.pyplot as plt

data = [10, 12, 14, 15, 18, 19, 20, 21, 35]

plt.boxplot(data)
plt.show()
```

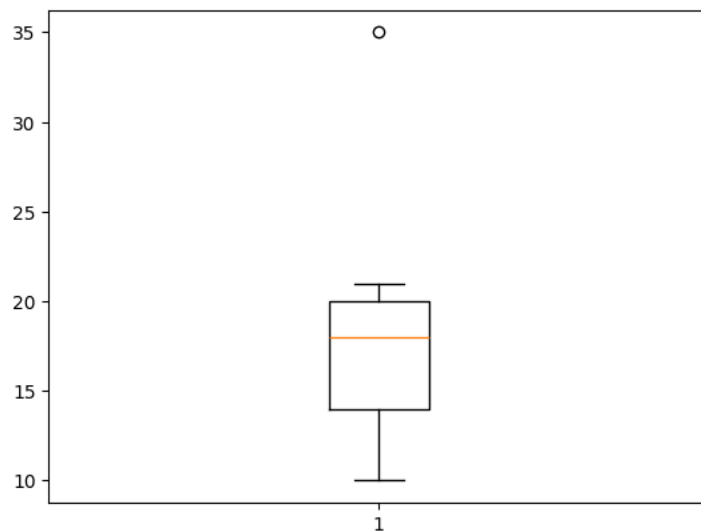


Figure 7. Basic box plot summarizing nine numerical observations.

Accessibility-Aware Version:

```
import matplotlib.pyplot as plt

algorithm_runtime = [10, 12, 14, 15, 18, 19, 20, 21, 35]

plt.figure(figsize=(7, 5))

plt.boxplot(
    algorithm_runtime,
    tick_labels=["Runtime"],
    patch_artist=True
)
```

```
plt.ylabel("Execution Time (ms)", fontsize=14)
plt.title("Distribution of Algorithm Execution Time", fontsize=16)

plt.grid(True)
plt.tight_layout()
plt.show()
```

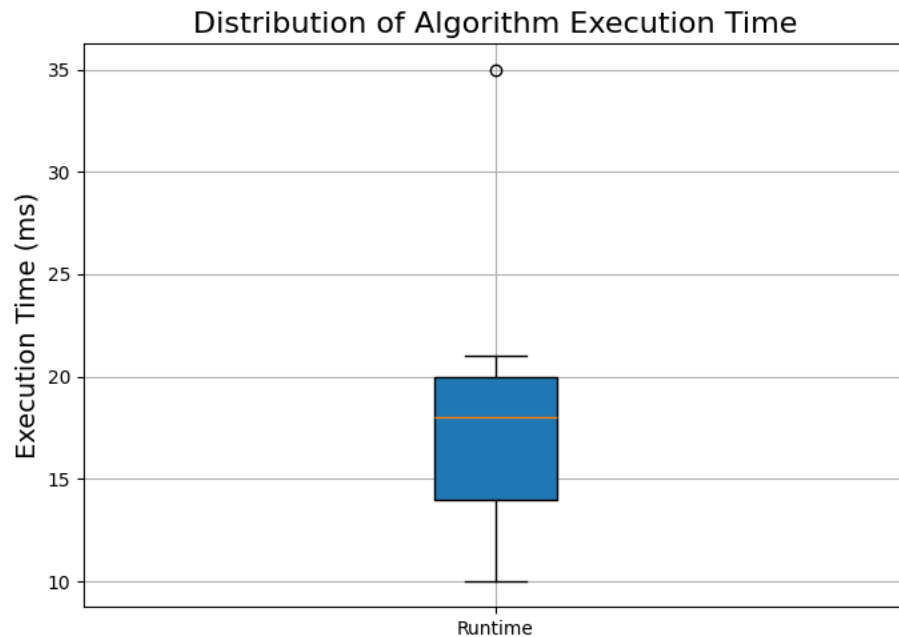


Figure 8. Accessibility-aware box plot of algorithm execution times.

The improved version:

- contextualizes the data,
- labels measurement units,
- improves readability,
- and supports interpretation of variability.

Example semantic interpretation: The box plot summarizes the execution time of an algorithm measured in milliseconds. The median runtime is approximately 18 milliseconds. Most execution times fall within a relatively narrow range, but one higher value near 35 milliseconds appears as a potential outlier, suggesting occasional slower executions.

5.5 Heatmaps (Matrix Relationships)

Heatmaps represent matrix values, correlations, intensity or magnitude, and structured comparisons.

Basic Example:

```
import matplotlib.pyplot as plt
import numpy as np
```

```
data = np.array([
    [1.0, 0.8, 0.2],
    [0.8, 1.0, 0.4],
    [0.2, 0.4, 1.0]
])
```

```
plt.imshow(data)
plt.show()
```

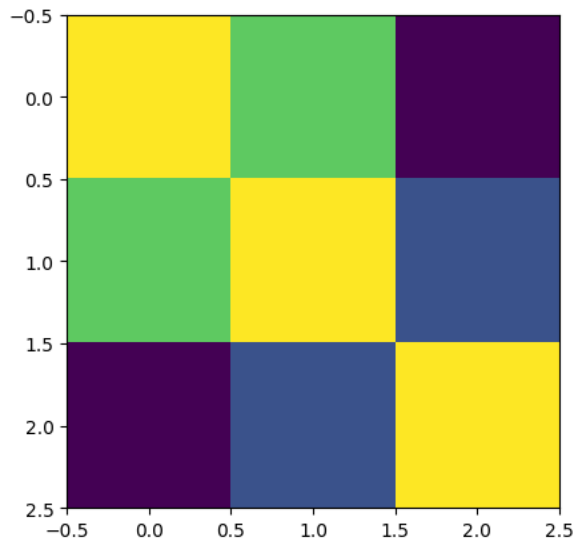


Figure 9. Basic heatmap of a 3-by-3 numerical matrix without semantic labels.

Problems:

- no labels,
- inaccessible default colormap,
- no explanation of matrix meaning,
- poor interpretability.

Accessibility-Aware Version:

```
import matplotlib.pyplot as plt
import numpy as np
```

```
correlation_matrix = np.array([
    [1.0, 0.8, 0.2],
    [0.8, 1.0, 0.4],
    [0.2, 0.4, 1.0]
])
```

```
feature_names = ["Study Time", "Attendance", "Exam Score"]
```

```

plt.figure(figsize=(6, 5))

heatmap = plt.imshow(
    correlation_matrix,
    cmap="cividis"
)

plt.xticks(range(len(feature_names)), feature_names, rotation=15)
plt.yticks(range(len(feature_names)), feature_names)

plt.title("Correlation Between Student Performance Variables\n",
fontsize=16)

plt.colorbar(heatmap, label="Correlation Strength")

plt.tight_layout()
plt.show()

```

Correlation Between Student Performance Variables

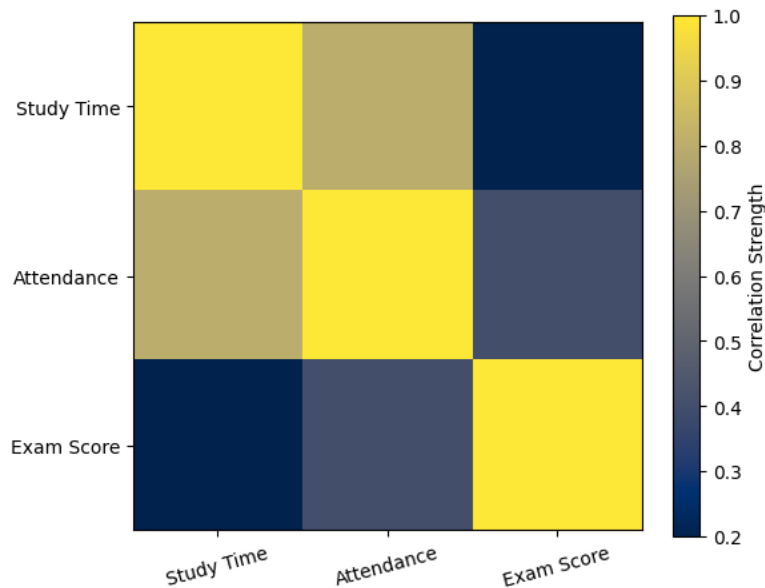


Figure 10. Accessibility-aware correlation heatmap for study time, attendance, and exam score.

The improved version:

- uses a colorblind-friendly colormap,
- labels rows and columns,
- includes semantic feature names,
- adds a labeled color scale,
- and improves contextual understanding.

Example semantic interpretation: The heatmap displays correlations between study time, attendance, and exam scores. Strong positive correlations exist between study time and exam scores, as well as between attendance and study time. The weakest relationship appears between attendance and exam scores, though the correlation remains positive overall.

6. Accessibility Tools for Code-Generated Graphics

Several emerging tools support the creation of accessible computational graphics directly from code. These tools should be integrated into educational workflows whenever possible.

6.1 MatplotAlt

MatplotAlt is a Python library that automatically generates alternative text descriptions for Matplotlib figures. The generated descriptions can:

- appear directly in Jupyter notebooks,
- be embedded into PNG metadata,
- or be exported into separate text files.

MatplotAlt supports semantic figure summaries, automatic extraction of axis information, and plot interpretation assistance. It can help instructors generate baseline descriptions, it encourages accessibility-aware workflows, and supports iterative refinement of descriptions.

MatplotAlt documentation, including usage examples, can be found here: <https://matplotalt.readthedocs.io/en/latest/index.html>.

Example:

```
import matplotlib.pyplot as plt
from matplotalt import show_with_alt

input_values = [1, 2, 3, 4]
squared_values = [1, 4, 9, 16]

plt.figure(figsize=(6, 4))

plt.plot(
    input_values,
    squared_values,
    color="black",
    marker="o",
    linewidth=2,
    label="Quadratic relationship"
)
```

```
plt.xlabel("Input value (x)", fontsize=12)
plt.ylabel("Squared output value (x2)", fontsize=12)
plt.title("Quadratic Growth Function: y = x2", fontsize=14)

plt.grid(True)
plt.legend()

plt.tight_layout()
show_with_alt(methods="markdown")
```

Output:

A line plot titled 'quadratic growth function: $y = x^2$ '. Input value (x) is plotted on the x-axis from 0.5 to 4.5 and squared output value (x^2) is plotted on the y-axis from 0 to 18, both using linear scales. Quadratic relationship is plotted in black. Quadratic relationship has a minimum value of $y=1$ at $x=0.5$, a maximum value of $y=16$ at $x=2$, and an average of $y=7.5$. data table:

input value (x)	squared output value (x ²)
1	1
2	4
3	9
4	16

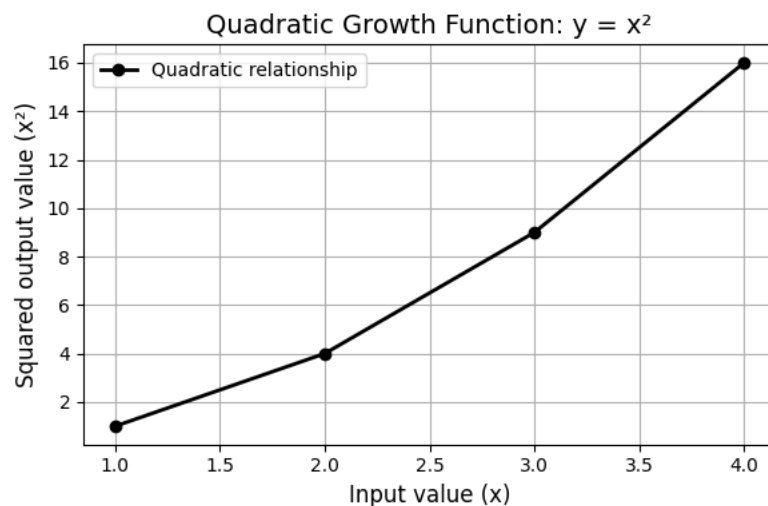


Figure 11. Quadratic-growth line graph used to demonstrate automatic descriptions with MatplotAlt.

Automated alt-text tools such as MatplotAlt are only as effective as the semantic richness of the plotting code. It should be used as a supplementary accessibility layer, not a replacement for semantic plotting practice, since it represents an interpretation tool that only reflects what is already encoded in the figure. Accessibility is not added after plotting but must be encoded during plotting.

6.2 Highcharts for Python

Highcharts for Python is an interactive visualization library designed for web-based data representation with strong built-in accessibility support. Unlike static plotting libraries such as Matplotlib, Highcharts prioritizes keyboard navigation, structured data exposure, and multimodal output, making it particularly suitable for blind and visually impaired users in interactive learning environments.

Highcharts for Python generates visualizations that are inherently accessible through multiple parallel representations:

- an interactive chart interface,
- a hidden structured data table,
- an automatically generated text summary,
- and optional sonification of data trends.

These layers allow users to explore data through non-visual navigation methods, including keyboard input and screen readers.

Highcharts for Python documentation and usage examples can be found here: <https://www.highcharts.com/integrations/python/>.

Example:

```
from highcharts_core.chart import Chart
from highcharts_core.options import HighchartsOptions

# Data
study_hours = ["1", "2", "3", "4", "5"]
exam_scores = [50, 60, 72, 85, 95]

options = HighchartsOptions()

options.chart = {"type": "line"}
options.title = {"text": "Study Time vs Exam Performance"}

options.x_axis = {
    "categories": study_hours,
    "title": {"text": "Study Hours"}
}

options.y_axis = {
    "title": {"text": "Exam Score (%)" }
}

options.series = [{
    "name": "Exam Score",
    "data": exam_scores
```

```

}]

options.accessibility = {"enabled": True}
options.sonification = {"enabled": True}

chart = Chart.from_options(options)

chart.download_chart(filename = 'accessible_chart.png',
                    format = 'png')
chart.download_chart(filename = 'accessible_chart.pdf',
                    format = 'pdf')
# to a file (and as a string)
chart.to_js_literal(filename = 'accessible_chart.js')

```

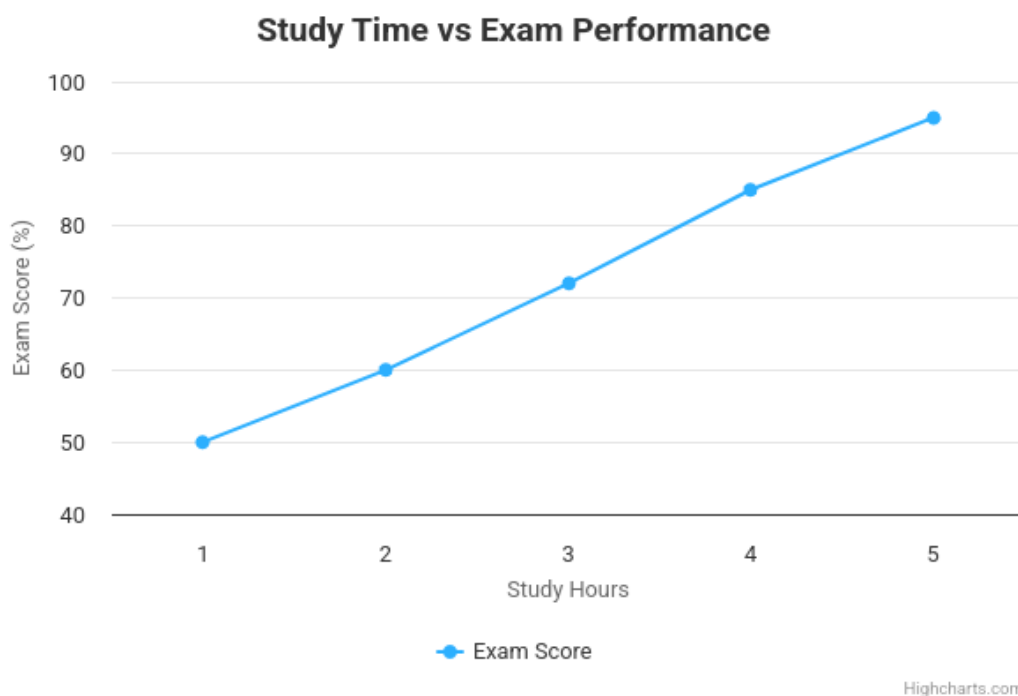


Figure 12. Highcharts example relating study hours to exam scores with accessibility features enabled.

6.3 Strauss (Sonification)

Strauss is a sonification framework for representing data as sound rather than visual plots. Instead of encoding information into axes, lines, or shapes, data is mapped into auditory dimensions such as pitch (frequency), volume (amplitude), duration, timbre, spatialization (left/right stereo position), and temporal sequencing.

This approach is particularly relevant for blind and visually impaired students because it removes dependence on visual interpretation entirely. Unlike traditional plotting libraries, Strauss does not produce a visual artifact by default. Instead, it produces a listenable representation of data structure and trends.

A correct accessibility-oriented implementation must ensure that every data dimension has a clearly defined auditory mapping, and that the mappings are consistent across datasets. Furthermore, every dataset must specify:

- which variable controls pitch or volume
- which variable controls time progression
- whether values are normalized or scaled

Basic flow of the Strauss code, from input data to output audio, is displayed below:

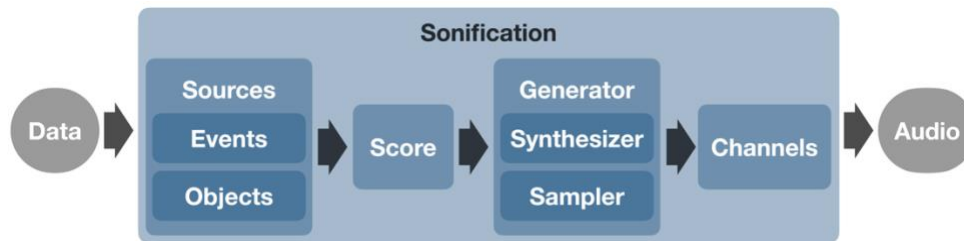


Figure 13. Schematic diagram illustrating the modular workflow of a strauss sonification; going all the way from the raw input data to the output audio representation.

Strauss for Python documentation and usage examples can be found at the following link: <https://strauss.readthedocs.io/en/latest/index.html>.

Example:

```
import matplotlib.pyplot as plt
from strauss.sonification import Sonification
from strauss.sources import Objects
from strauss import channels
from strauss.score import Score
from strauss.generator import Synthesizer
import IPython.display as ipd
import os
from scipy.interpolate import interp1d
import numpy as np

np.random.seed(0)

N = 300
x = np.linspace(0,1,N)
y = np.zeros(N)

gauss = lambda x, m, s: np.exp(-(x-m)**2/s)
for i in range(10):
    a,b,c = np.random.random(3)
    y += gauss(x, b, 1e-3*c) * a ** 3

y /= y.max()*1.2
```

```

y += 0.15

plt.plot(x,y)
plt.ylabel('Some dependent Variable')
plt.xlabel('Some independent Variable')
plt.show()

# Setup universal sonification parameters
system = "stereo" # Specify audio system
length = 15. # Length of the sonification in s

generator = Synthesizer()
generator.load_preset('pitch_mapper')
generator.preset_details('pitch_mapper')

```

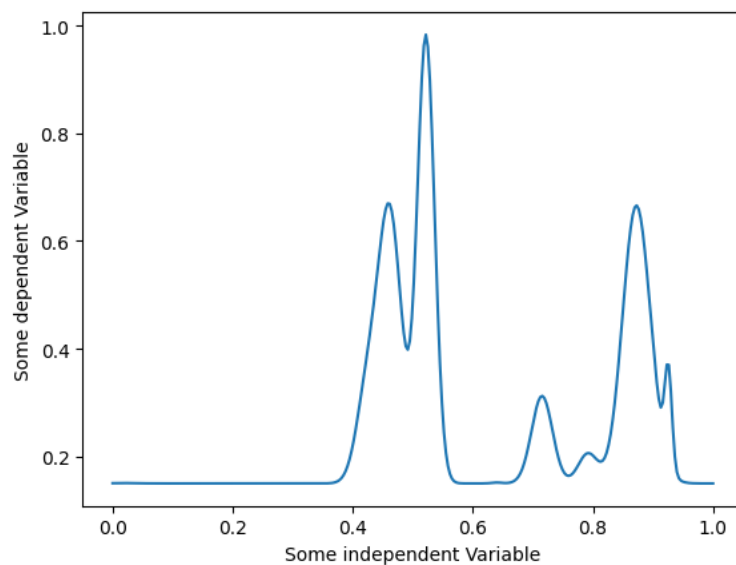


Figure 14. Numerical source data used for the STRAUSS sonification examples.

Pitch Mapping

```

notes = [["A2"]]
score = Score(notes, length)

data = {'pitch':1.,
        'time_evo':x,
        'azimuth':(x*0.5+0.25) % 1,
        'polar':0.5,
        'pitch_shift':y**0.7}

# set up source
sources = Objects(data.keys())
sources.fromdict(data)

```

```

sources.apply_mapping_functions()

soni = Sonification(score, sources, generator, system)
soni.render()
soni.notebook_display()

```

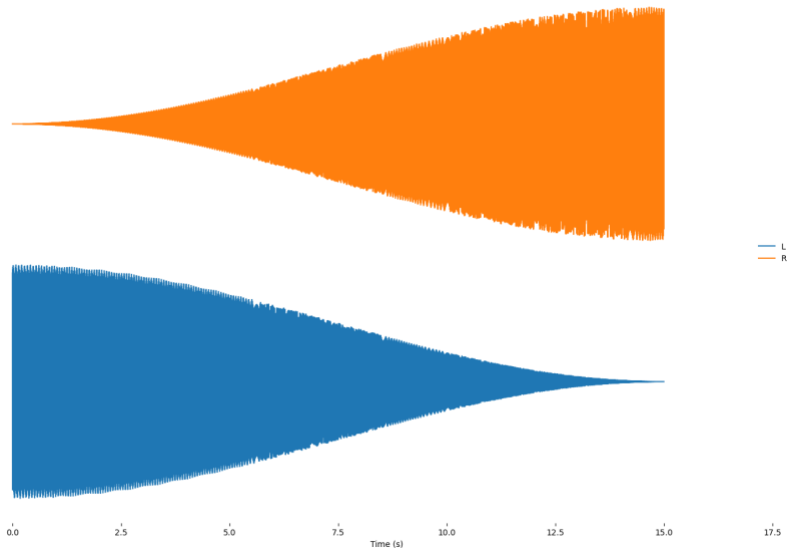


Figure 15. Rendered stereo audio output from the STRAUSS pitch-mapping example.

Volume Mapping

```

notes = [["A2"]]
score = Score(notes, length)

data = {'pitch':1.,
        'time_evo':x,
        'azimuth':(x*0.5+0.25) % 1,
        'polar':0.5,
        'volume':y**0.7}

# set up source
sources = Objects(data.keys())
sources.fromdict(data)
sources.apply_mapping_functions()

soni = Sonification(score, sources, generator, system)
soni.render()
soni.notebook_display()

```

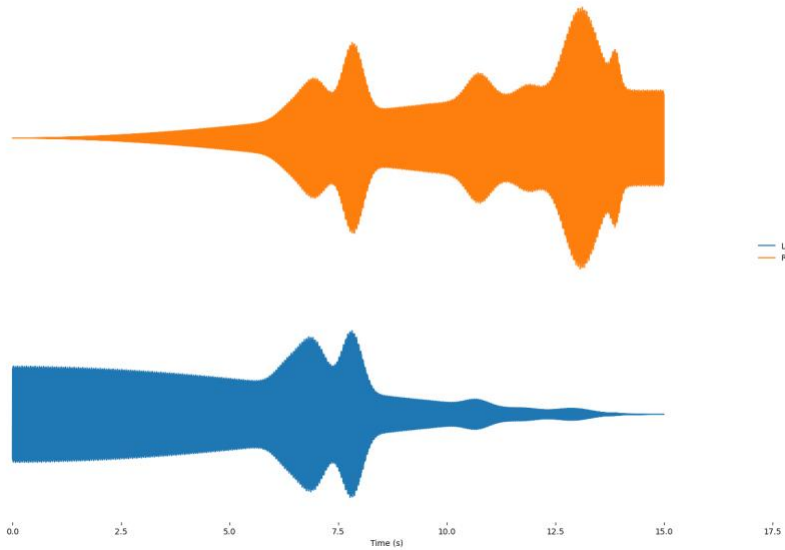


Figure 16. Rendered stereo audio output from the STRAUSS volume-mapping example.

Filter Cutoff Mapping

```
generator = Synthesizer()
generator.modify_preset({'filter':'on'})

notes = [["C2", "G2", "C3", "G3"]]
score = Score(notes, length)

data = {'pitch':[0,1,2,3],
        'time_evo':[x]*4,
        'azimuth':[(x*0.5+0.25) % 1]*4,
        'polar':[0.5]*4,
        'cutoff':[y**0.8]*4}

# set up source
sources = Objects(data.keys())
sources.fromdict(data)
sources.apply_mapping_functions()

soni = Sonification(score, sources, generator, system)
soni.render()
soni.notebook_display()
```

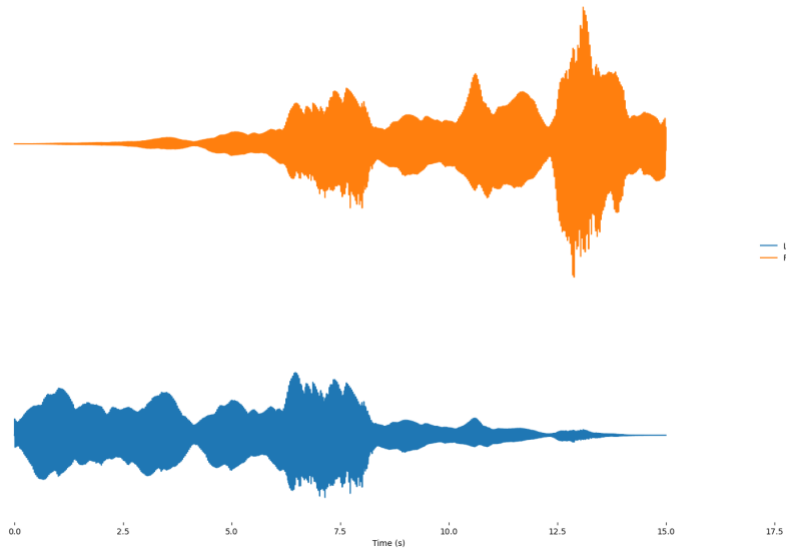


Figure 17. Rendered stereo audio output from the STRAUSS filter-cutoff mapping example.

STRAUSS is a sonification framework where accessibility is not added after visualization but defined through explicit mappings between data variables and auditory channels using a Score-based architecture.

6.4 Chatlas with Gemini

Chatlas is an accessibility-oriented framework that integrates large language models (such as Gemini) into data visualization workflows. Its primary function is to augment code-generated visualizations with structured natural-language explanations, interpretations, and accessibility summaries.

Unlike Matplotlib (which is rule-based) or STRAUSS (which is auditory encoding), Chatlas operates as a semantic interpretation layer, translating charts, datasets, and code outputs into human-understandable descriptions.

It is particularly useful in Computer Science education for:

- generating accessible explanations of plots and diagrams,
- supporting blind students with conceptual understanding,
- providing step-by-step interpretation of visual structures,
- and enabling conversational exploration of data.

Its documentation can be found at the following link: <https://posit-dev.github.io/chatlas/>.

Example Workflow:

```
from chatlas import content_image_file, ChatGoogle
import matplotlib.pyplot as plt

chat = ChatGoogle(model="gemini-2.5-flash", api_key="YOUR_API_KEY")
def save_ai_alt_text(fig, base_name="alttext_report"):
```

```

# 2. Save the plot locally
img_path = f"{base_name}.png"
fig.savefig(img_path)

# 3. Ask the AI to describe the image using chatlas helper
# We use chat.chat() to get a direct response
response = chat.chat(
    "Describe this chart for a visually impaired user.",
    content_image_file(img_path)
)

# 4. Extract text and save to .txt
alt_text = str(response)
with open(f"{base_name}.txt", "w") as f:
    f.write(alt_text)

print(f"Analysis complete. Created {base_name}.txt")
return alt_text

categories = ["Algorithm A", "Algorithm B"]
performance = [10, 20]

fig, ax = plt.subplots(figsize=(6, 4))

bars = ax.bar(categories, performance, color="black")

# Accessibility-oriented labeling
ax.set_title("Performance Comparison of Two Algorithms")
ax.set_xlabel("Algorithm")
ax.set_ylabel("Execution Time (ms)")

# Improve readability
ax.grid(axis="y", linestyle="--", alpha=0.5)

# Add value labels (important for blind + low-vision accessibility)
for bar in bars:
    height = bar.get_height()
    ax.text(
        bar.get_x() + bar.get_width() / 2,
        height,
        f"{height}",
        ha="center",
        va="bottom"
    )

```

```
plt.tight_layout()
plt.show()
save_ai_alt_text(fig)
```

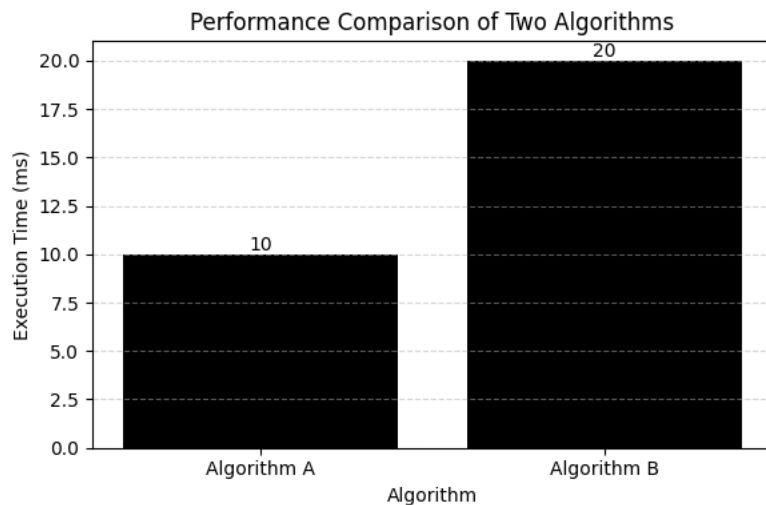


Figure 18. Algorithm execution-time comparison used in the Chatlas + Gemini alt-text example.

This is a bar chart titled "Performance Comparison of Two Algorithms." It displays the execution time in milliseconds (ms) for two different algorithms.

Chart Details:

- **Horizontal Axis (X-axis):** Represents the 'Algorithm', with two categories: 'Algorithm A' and 'Algorithm B'.
- **Vertical Axis (Y-axis):** Represents 'Execution Time' in milliseconds (ms). The scale ranges from 0 to 20 ms, with major tick marks every 2.5 milliseconds.

Data Presented:

- **Algorithm A:**
 - A bar corresponding to 'Algorithm A' shows an execution time of **10 ms**.
 - The numerical value "10" is displayed directly above this bar.
- **Algorithm B:**
 - A bar corresponding to 'Algorithm B' shows an execution time of **20 ms**.
 - The numerical value "20" is displayed directly above this bar.

Summary and Comparison: Algorithm A has an execution time of 10 milliseconds, while Algorithm B has an execution time of 20 milliseconds. This indicates that Algorithm B takes twice as long to execute as Algorithm A. Therefore, **Algorithm A demonstrates better performance** with a shorter execution time compared to Algorithm B.

7. Diagram-to-Text Conversion and Accessible Diagram Creation with PlantUML

Computer Science teaching often relies on diagrams to explain systems, software architecture, algorithms, workflows, object-oriented relationships, and computational processes. These

diagrams are commonly presented visually through UML models, flowcharts, network diagrams, or state machines. While these representations are very useful in technical education, they often present accessibility barriers for blind and visually impaired students when distributed only as static images (PNG, PDF, or SVG files without semantic structure).

PlantUML offers an alternative because it approaches diagram construction as a textual activity rather than a purely graphical one. Instead of drawing boxes and arrows manually using drag-and-drop interfaces, authors describe diagrams using plain text that is later rendered into a visual representation.

Therefore, the primary accessibility advantage of PlantUML is that the source representation itself is readable by screen readers. Students can navigate the diagram definition line by line using standard text editors and keyboard navigation tools. Relationships, entities, and interactions are encoded explicitly in the syntax rather than implied solely through spatial placement.

For example:

```
@startuml
```

```
User --> Database : Query
```

```
@enduml
```

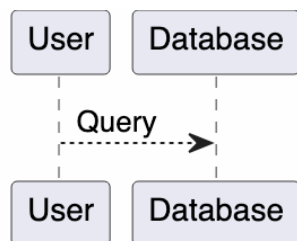


Figure 19. Simple PlantUML interaction from a User to a Database.

already contains several semantically meaningful components that can be interpreted sequentially by a screen reader. The source entity (“User”), the target entity (“Database”), and the action (“Query”) are explicitly represented in text. This sequential representation is easier to follow than interpreting arrows and shapes within a complex visual layout.

Teachers can explain systems directly from the source code, building the diagram step by step while verbally describing each interaction or relationship. This can reduce the cognitive burden associated with immediately interpreting a dense visual representation. In sequence diagrams especially, the textual structure mirrors the chronological flow of interactions, making it suitable for screen-reader-based exploration.

PlantUML additionally supports exporting diagrams into formats such as SVG, LaTeX, and ASCII-like textual representations. In some situations, these formats may offer better accessibility than raster images. However, it is important to recognize that the rendered outputs remain primarily visual artifacts. Once exported as PNG or SVG, accessibility again depends on alternative text, semantic SVG structure, screen-reader compatibility, and external descriptions. PlantUML

therefore improves accessibility at the authoring and structural level but does not automatically guarantee accessible final outputs.

PlantUML documentation can be found at the following link: <https://plantuml.com/>.

This guideline focuses on encouraging accessibility-aware diagram creation practices where semantic structure, textual representations, and non-visual exploration are integrated directly into the process of creating Computer Science teaching materials.

7.1 General Accessibility Guidelines for PlantUML Diagrams

1. Prioritize Semantic Naming

To support accessibility effectively, PlantUML diagrams should be written using meaningful and descriptive identifiers. Abstract expressions such as `A --> B : Node1` provide little semantic value when read aloud through a screen reader. Instead, entities and interactions should communicate their functional role clearly. A relationship such as:

`Student --> AuthenticationServer : Login request`

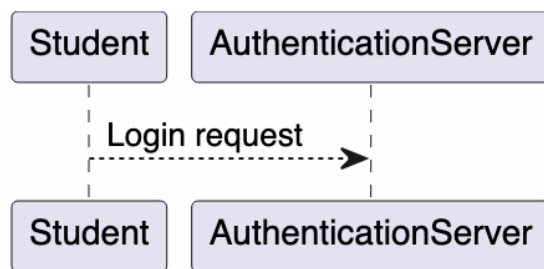


Figure 20. PlantUML interaction using meaningful entity and action names.

is significantly easier to interpret than an unnamed or symbolic connection because the interaction itself carries semantic meaning.

2. Prefer Simple, Modular Diagrams

Complex systems should also be divided into smaller conceptual units whenever possible. Very large diagrams containing dozens of entities and interactions may be difficult to interpret both visually and non-visually. Modularizing diagrams into smaller subsystems or progressive stages can support more effective learning and reduce cognitive overload.

3. Use Structured Interaction Ordering

PlantUML code should generally be accompanied by a structured explanatory narrative. The textual syntax alone does not communicate the full pedagogical intention of the diagram. Students benefit from explicit explanations describing:

- the purpose of the system,
- the role of each entity,
- the interaction flow,

- and the key conceptual takeaway.

4. Supplement with Structured Textual Explanation

Although the PlantUML source already exposes the interaction structure textually, an accompanying explanation should provide additional context. In the example provided above, the textual explanation could be:

The diagram represents a client-platform-database interaction pattern in which a student requests course materials, the platform retrieves information from a database, and accessible resources are returned to the student. The diagram demonstrates a common layered architecture used in web systems and educational platforms.

5. Consider the Limitations

PlantUML also has important limitations. It is not designed for mathematical plots, geometric reasoning, or continuous spatial visualization. It does not automatically generate semantic explanations or pedagogically appropriate descriptions. Furthermore, some diagrammatic concepts still rely heavily on spatial reasoning, grouping, or layout intuition that may not be fully captured through sequential text alone. For this reason, PlantUML should be viewed as one component within a broader accessibility framework rather than as a complete solution by itself.

7.2 Sequence Diagrams

Sequence diagrams are often among the most accessible UML diagram types because their structure naturally follows chronological order. Interactions are represented sequentially from top to bottom, which aligns well with screen-reader navigation and verbal explanation.

However, accessibility problems may still occur when:

- too many participants are included,
- interaction labels are vague,
- or the diagram contains dense bidirectional communication.

Basic Example:

```
@startuml
A -> B : msg
B -> C : req
C --> B : ok
@enduml
```

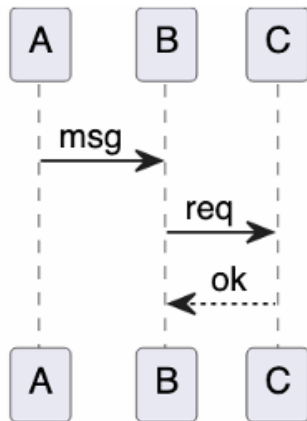


Figure 21. Basic sequence diagram using abstract participant and message names.

A less accessible example may use short identifiers without semantic meaning. Although technically valid, this diagram provides little educational context when read aloud.

Accessibility-Aware Version:

@startuml

actor Student

participant LearningPlatform

database CourseDatabase

Student -> LearningPlatform : Request course materials

LearningPlatform -> CourseDatabase : Query available resources

CourseDatabase --> LearningPlatform : Return resource list

LearningPlatform --> Student : Display accessible materials

@enduml

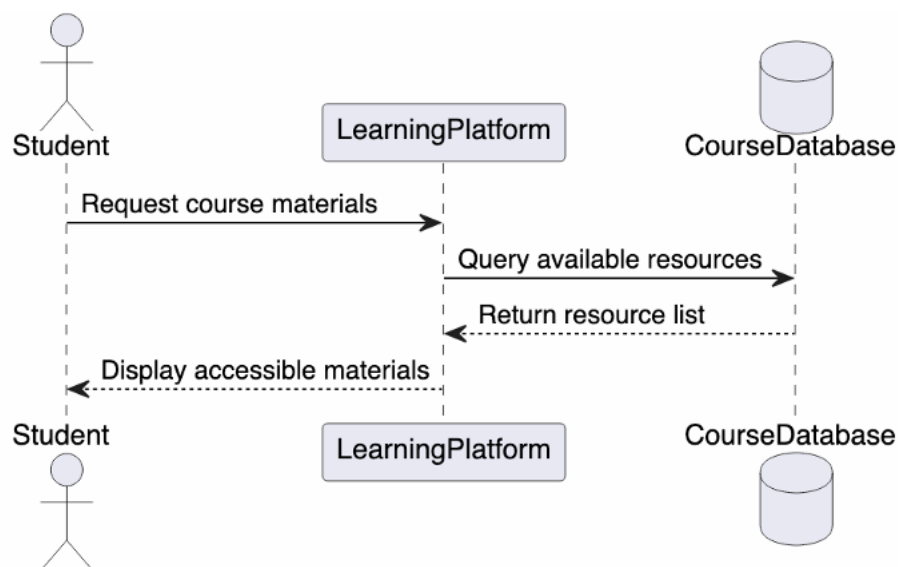


Figure 22. Accessibility-aware sequence diagram for retrieving accessible course materials.

The improved version:

- uses meaningful entity names,
- describes actions explicitly,
- follows a clear interaction order,
- and supports screen-reader interpretation.

Example semantic explanation:

The diagram demonstrates a client-service-database interaction pattern commonly used in web systems. It represents the process of retrieving course materials from an educational platform.

Entities:

- **Student**: initiates the request
- **LearningPlatform**: processes user requests
- **CourseDatabase**: stores educational resources

Interaction flow:

- The student requests course materials.
- The platform queries the database.
- The database returns available resources.
- The platform delivers accessible materials back to the student.

7.3 Class Diagrams

Class diagrams present object-oriented structures such as classes, attributes, methods, inheritance, and associations. These diagrams can become difficult to interpret non-visually because visual layouts often communicate grouping and hierarchy implicitly.

Accessibility-aware class diagrams should therefore:

- use descriptive class names,
- organize relationships clearly,
- avoid excessive inheritance depth,
- and include textual explanations of relationships.

Basic Example:

```
@startuml
class A
class B
A --|> B
@enduml
```

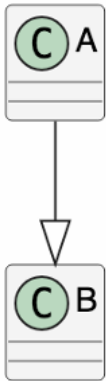


Figure 23. Basic class diagram showing inheritance between abstract classes A and B.

The diagram technically represents inheritance but does not explain what the entities mean.

Accessibility-Aware Version:

@startuml

```

class User {
  +username : String
  +login()
}
  
```

```

class Student {
  +studentID : Integer
  +accessCourseMaterials()
}
  
```

User <|-- Student

@enduml

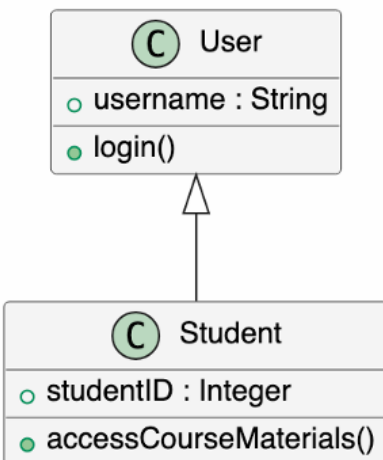


Figure 24. Accessibility-aware class diagram showing Student inheriting from User.

The improved version:

- uses meaningful class names,
- exposes attributes and methods explicitly,
- clarifies inheritance structure,
- and supports sequential reading.

Example semantic interpretation: The diagram represents an object-oriented inheritance relationship. The class Student inherits from the class User. This means that Student contains all properties and behaviors defined in User, including the username attribute and login functionality, while also introducing student-specific features such as a student identifier and access to course materials.

7.4 Activity Diagrams

Activity diagrams represent workflows, algorithms, or procedural logic. Accessibility problems often emerge when complex branching structures rely heavily on spatial positioning.

Accessibility-aware activity diagrams should:

- use clearly labeled actions,
- minimize unnecessary branching,
- and describe decision logic explicitly.

Basic Example:

@startuml

```
start
:Input;
if (Valid?) then (Yes)
  :Process;
else
  :Reject;
endif
stop
```

@enduml

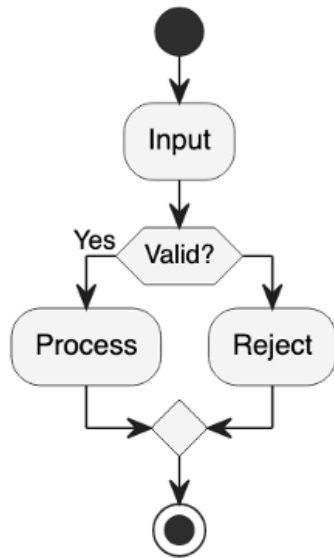


Figure 25. Basic activity diagram with a generic validation decision.

The decision process is present but lacks semantic context.

Accessibility-Aware Version:

@startuml

start

:Student submits assignment;

if (Assignment format valid?) then (Yes)

:Run plagiarism check;

:Store assignment in submission system;

else (No)

:Display formatting error message;

endif

stop

@enduml

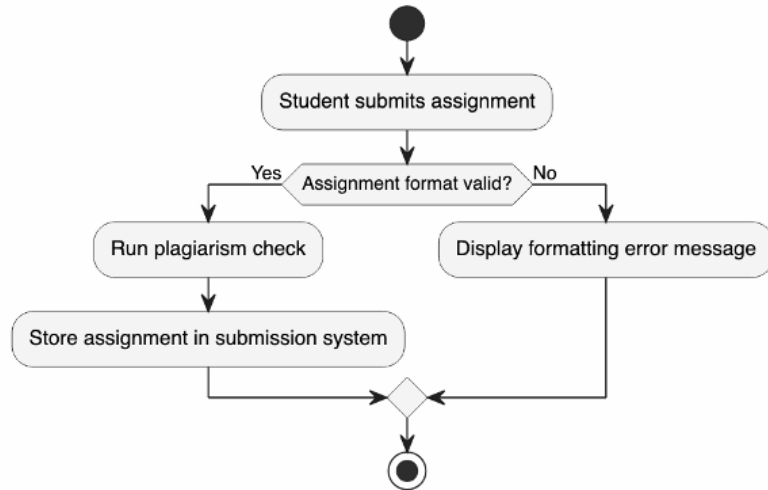


Figure 26. Accessibility-aware activity diagram for validating and processing a student assignment.

The improved version:

- clarifies workflow purpose,
- uses descriptive decisions,
- supports step-by-step narration,
- and reduces ambiguity.

Example semantic interpretation: The activity diagram represents the process of submitting a student assignment. After submission, the system checks whether the file format is valid. If the format is correct, the assignment is processed and stored. Otherwise, the system returns an error message requesting correction.

7.5 State Diagrams

State diagrams represent changes between system states triggered by events. These diagrams are useful in operating systems, embedded systems, networking, and software engineering.

Accessibility-aware state diagrams should:

- clearly define each state,
- describe triggering events explicitly,
- avoid overly dense transition networks.

Accessibility-Aware Example:

@startuml

[*] --> LoggedOut

LoggedOut --> Authenticating : User submits login

Authenticating --> LoggedIn : Credentials valid

Authenticating --> LoggedOut : Authentication failed

LoggedIn --> LoggedOut : User logs out

@enduml

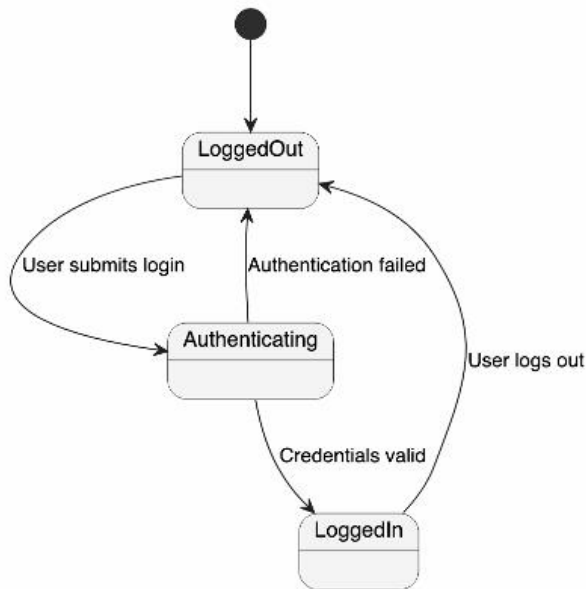


Figure 27. State diagram describing the transitions in a user-authentication process.

The improved version:

- uses meaningful state names,
- describes transitions semantically,
- and supports linear interpretation.

Example semantic interpretation: The state diagram models a user authentication system. The system begins in the LoggedOut state. When the user submits login credentials, the system enters the Authenticating state. Successful authentication moves the user into the LoggedIn state, while failed authentication returns the system to LoggedOut.

7.6 Network and Graph Diagrams

Graph and network diagrams are commonly used in cybersecurity, distributed systems, networking, and graph algorithms. These diagrams often rely heavily on spatial layouts, which can create accessibility barriers.

Accessibility-aware graph diagrams should:

- explicitly describe node relationships,
- avoid unnecessary visual complexity,
- provide adjacency-style textual explanations.

Accessibility-Aware Example:

@startuml

node Client
node WebServer
node Database

Client --> WebServer : HTTP request
WebServer --> Database : SQL query

@enduml

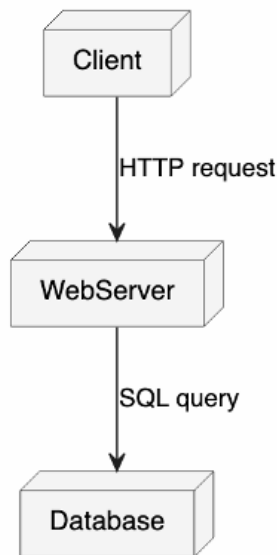


Figure 28. Simple three-tier client–web-server–database network architecture.

Example semantic interpretation: The network diagram represents a simple client-server architecture. A client sends HTTP requests to a web server. The web server processes requests and communicates with a database using SQL queries. The diagram demonstrates a three-tier architecture commonly used in web applications.

7.7 Accessible Teaching Strategies Using PlantUML

PlantUML can support accessibility-oriented teaching workflows.

- 1. Code-First Diagram Instruction:** Teachers can introduce relationships through text first, explain interactions verbally, and render diagrams afterward.
- 2. Step-by-Step Diagram Construction:** Instead of presenting a complete complex diagram immediately, the teacher can build diagrams incrementally, while narrating each added component and explaining the relationships progressively.
- 3. Screen-Reader-Compatible Editing:** Students can create diagrams using standard text editors, navigate syntax using keyboard shortcuts, and participate in collaborative modeling activities.

7.8 Alternative Text-Based Diagram Systems

Related text-based diagram systems such as DOT/Graphviz and Mermaid can be used as alternatives and complementary tools. These systems share a similar philosophy in which diagrams are authored through structured text rather than direct manipulation of graphical objects.

DOT/Graphviz is useful for creating graph structures, dependency graphs, and network relationships. It uses declarative graph syntax, structured edge definitions, and can be integrated with computational workflows. The documentation can be accessed at the following link: <https://graphviz.org/documentation/>.

Mermaid is a lightweight text-based diagram language with strong integration into GitHub, Markdown, and documentation systems. Its syntax is easily readable, it supports web-native workflows and has collaborative documentation support. Its documentation can be found at the following link: <https://mermaid.ai/docs/>